



La méthode Avaliance

Travailler avec l'IA sans lui faire confiance : des portes qui refusent, des témoins qu'on a vus rouges, et une page que l'on ouvre sans lire le code.

-
1. Une convention se respecte tant qu'on y pense ; une **porte**, non.
 2. Un test vert ne prouve rien tant qu'on ne l'a pas vu **rouge**.
 3. Pas de preuve, pas de constat.

Ce que vous avez entre les mains

Une méthode de travail avec l'IA, éprouvée sur de vrais projets en production. **Chaque règle qu'elle contient a été payée une fois.** C'est la seule chose qui distingue ce document d'un cours : vous ne trouverez pas ici de bonne pratique inventée pour l'exemple — mais des dates, des montants et des pannes.

Trois étages, et on ne les mélange jamais. Ce qui est à nous est livré. Ce qui est aux autres est recommandé, jamais distribué. Ce qui est déjà dans Claude Code n'a pas à être installé — et c'est le plus sous-estimé des trois.

1 · Les trois étages	p. 3
2 · Installer — deux commandes, aucun script	p. 4
3 · Les quatre premières semaines	p. 5
4 · Les 9 kits	p. 6
5 · Les 19 skills	p. 7
6 · Les principes, et la panne derrière chacun	p. 8
7 · Le hub — la page que l'on montre	p. 10
8 · Ce qui est déjà dans Claude Code	p. 11
9 · Ce qui n'est pas à nous — et ce qu'il faut en retenir	p. 12
10 · Quand ça ne marche pas	p. 13

À qui ce document est destiné

VOUS ÊTES	VOUS LISEZ
Développeur, vous utilisez déjà l'IA pour écrire du code	tout, dans l'ordre. 🚫 Et vous ne sautez pas le chapitre 3
Responsable technique, vous équipez une équipe	les chapitres 1, 6 et 7 d'abord
Non technique, on vous a livré un projet	★ le chapitre 7 seul : le hub est fait pour vous

🚫 **Ce qu'on ne vous promet pas.** Du « tout automatique ». Ce qui est livré ici est **automatisé avec validation humaine** — et la différence compte le jour où ça se trompe.

1 · Les trois étages

ÉTAGE	CE QUE C'EST	COMMENT ÇA VOYAGE
1 · À NOUS	8 kits, 19 skills, les portes, le hub	★ livré dans ce plugin
2 · RECOMMANDÉ	des dépôts GitHub d'autres personnes	🚫 jamais livré : un lien, une licence, et la décision à copier
3 · DÉJÀ LÀ	ce qui est fourni avec Claude Code	★ rien à installer — seulement à connaître

Pourquoi l'étage 2 n'est jamais embarqué. Ce qui n'est pas à nous n'est pas à nous. On en recommande la lecture, on n'en distribue pas le code — et un paquet qui embarque du code sans sa licence est un problème juridique, pas un détail.

La règle qui vaut pour les trois

Avant d'installer quoi que ce soit, on vérifie dans cet ordre :

#	LA QUESTION	OÙ L'ON REGARDE
1	Est-ce déjà dans Claude Code ?	<code>/help</code> , <code>/plugin</code>
2	Est-ce dans nos kits ?	<code>/catalogue</code>
3	Un dépôt porte-t-il la décision dont j'ai besoin ?	chapitre 9 — et on copie la décision, pas le dépôt
4	Alors seulement : installer quelque chose de nouveau	—

🚫 Sauter cet ordre est la façon la plus courante de se retrouver avec quatre outils qui font la même chose — et aucun qu'on maîtrise.

Skill ou kit ? La distinction tient en trois lignes

	UNE SKILL	UN KIT
C'est quoi	du texte , chargé dans une session	des fichiers , posés dans un projet
Ça vit où	chez vous, dans votre Claude	dans le dépôt du projet
Ça meurt quand	la session se ferme	jamais

2 · Installer — deux commandes, aucun script

Tout se fait **dans une session Claude Code**. 🚫 Pas de terminal, pas de PowerShell, pas de ligne de commande système.

Les deux commandes

```
# 1. ajouter la place de distribution
/plugin marketplace add ahmedbentaleb/workflow

# 2. installer le plugin
/plugin install avaliance@avaliance
```

Puis, une fois

```
/demarrer
```

★ Il pose **une seule question** — et c'est la seule que tout le monde sait répondre :

Qu'est-ce que vous démarrez, et dans quel métier ?

VOUS RÉPONDEZ	VOUS RECEVEZ	VOUS NE RECEVEZ PAS
Je développe	le socle de code, les portes, les tests, le suivi	—
Je fais du marketing	le socle non technique + la charte + les rapports	🚫 aucune porte de commit, aucun test
Je suis commercial	le socle non technique + les rapports	🚫 ni charte, ni tests
Autre chose	★ le socle non technique seul	🚫 aucun outil de code
J'audite un code	la méthode d'audit	🚫 et rien qui corrige

★ **Vous vous décrivez, vous ne choisissez pas un outil.** Choisir un outil demande de savoir ce qu'il fait — c'est le problème qu'on vient d'enlever.

★ Le socle non technique

Si vous n'écrivez pas de code, vous recevez **quatre endroits où écrire, et rien d'autre** :

OÙ	CE QU'ON Y MET	QUAND ON LE RELIT
Le plan	★ ce qui reste à faire, en haut	tous les jours
Les journaux	ce qui se passe, ce qui se décide, ce qui casse	« pourquoi on avait décidé ça ? »
Les décisions	une page, avec ce qu'on a écarté	quand quelqu'un veut la défaire
Les rapports	ce qu'un tiers lit, daté, jamais écrasé	chaque semaine

— **Aucune porte de commit, aucun banc de tests, aucun outil à installer.** Des fichiers dont on n'a pas l'usage ne se relisent pas — donc ils se périment, et on les suit quand même.

Ce qu'il faut sur la machine

OUTIL	CE QUI NE MARCHE PAS SANS LUI	BLOQUANT ?
<code>git</code>	aucune porte, aucun journal, aucune session mère/fille	— oui
<code>node</code>	le hub ne se génère pas : plus de page de contrôle	non — le reste marche

★ Un outil installé n'est **pas** visible de la session en cours : son `PATH` a été lu à l'ouverture. Si une commande reste introuvable, fermez et rouvrez Claude — — ne réinstallez pas.

Mettre à jour plus tard

`/plugin`

Onglet des places de distribution, puis mettre à jour. ★ Tout le monde passe à la même version en même temps : c'est le seul intérêt réel d'un plugin par rapport à un dossier copié.

Les trois modes sont actifs tout seuls. `focus`, `brainstorm` et `plan` ne se tapent pas. Ils règlent la forme des réponses, empêchent de coder quand vous réfléchissez à voix haute, et écrivent le plan dès trois décisions accumulées.

— **Ne posez aucun kit le premier jour.** Un dépôt qui porte des fichiers que personne n'utilise porte des fichiers que personne ne relit — donc qui se périment. Et un texte qui décrit un monde disparu est plus dangereux qu'un texte absent, parce qu'on le suit.

3 · Les quatre premières semaines

Une chose par semaine, et on la garde.

Semaine 1 — la forme, rien d'autre

Vous travaillez normalement. Vous devez ressentir trois choses :

CE QUI CHANGE	C'EST QUOI
Les réponses tiennent en quinze lignes	<code>focus</code> — vous ne décrochez plus au milieu
Claude ne code plus quand vous réfléchissez	<code>brainstorm</code>
Un <code>_ops/PLAN_<date>.md</code> apparaît sans le demander	<code>plan</code> , dès 3 points

★ **Pourquoi attendre pour le reste** : si la forme des réponses ne tient pas, vous ne lirez pas les rapports que la méthode produit — et tout le reste sera perdu.

Semaine 2 — une porte, et vous la voyez refuser

Choisissez **un** dépôt. Pas le plus important : le deuxième.

```
/poser-kit projet <votre dépôt>
```

Puis le geste de la semaine — il casse des choses exprès, dans une copie jetable, et regarde si les portes refusent :

```
sh _ops/sabotage/verifier-les-portes.sh
```

Une porte jamais vue refuser est une porte dont on ignore si elle refuse ou si elle dort. Trois minutes, et c'est irréfutable.

Semaine 3 — écrire pourquoi, pas quoi

```
sh _ops/dire.sh decisions "Postgres plutot que Mongo : on a besoin de jointures"
```

Et votre premier ADR — une fiche de décision — quand la réponse à cette question est oui :

Quelqu'un pourrait-il défaire ça dans six mois en croyant bien faire ?

❌ COMMENTAIRE INUTILE

✅ COMMENTAIRE QUI SERT

```
// incrémente le compteur
```

```
// sans ce compteur, deux sessions déployaient le même soir
```

★ C'est la semaine qu'on ne rattrape pas. Auditer et remettre à niveau se font après coup ; écrire *pourquoi* on a décidé, non. Dans six mois il ne reste que le code, et il ne dit jamais pourquoi.

Semaine 4 — le hub, et vous le montrez

```
node _ops/hub/generer-hub.mjs
```

Ouvrez `_ops/hub/index.html`, puis **montrez-le à quelqu'un qui ne code pas**.

Tant que la qualité est un chiffre dans une CI, elle n'existe que pour les développeurs. Dès qu'elle est une page, elle devient un sujet de réunion.

Après — un kit à la fois, quand le déclencheur arrive

LE KIT	LE DÉCLENCEUR, CHEZ VOUS
<code>donnees</code>	★ vous n'avez jamais essayé de restaurer une sauvegarde
<code>deploiement</code>	vous avez déjà cru une correction livrée alors qu'elle ne l'était pas
<code>couts</code>	vous avez une clé d'API et aucun plafond
<code>design</code>	on vous a demandé « c'est quoi vos couleurs ? »
<code>rapport</code>	on vous demande « où ça en est ? » plus d'une fois par semaine
<code>audit</code>	vous reprenez le code de quelqu'un d'autre

4 · Les 9 kits

Un kit pose des fichiers dans un projet. ★ **Il y a deux socles** — `base` pour tout ce qui n'est pas du code, `projet` pour du code. Les autres se posent par-dessus et refusent de s'installer sans socle.

```
/poser-kit <nom du kit> <chemin du projet>
```

KIT	CE QU'IL POSE	LA PANNE QUI L'A FAIT NAÎTRE
<code>base</code> <code>socle</code>	★ le socle non technique : plan, trois journaux, décisions, rapports	Tout parlait de code. Sur douze compétences utilisables sans coder, onze parlaient de dépôt, de commit ou de test — un commercial qui installait refermait.
<code>projet</code> <code>socle code</code>	le hub, les portes, le banc de tests, le sabotage, les trois journaux, les ADR	Six sessions dans le même dossier : trois <code>git add -A</code> ont emporté le travail des autres. Et un verrou de déploiement écrit en majuscules dans un document, ignoré de bonne foi par deux sessions le même soir.
<code>audit</code>	la méthode en onze axes, la fiche de constat, le registre à numérotation stable	Un rapport d'audit sans preuve localisée est contesté — et tout le rapport avec.
<code>design</code>	une charte vide à établir, les contraintes de figure	La charte d'un autre projet, posée par défaut, était suivie sans que personne l'ait décidé.
<code>déploiement</code>	la procédure à un seul endroit, le script qui prend le verrou, le témoin « le serveur sert »	Une correction crue livrée trois fois dans la même journée . À chaque fois quelque chose était vert.
<code>donnees</code>	migrations et leur retour arrière, accès, sauvegarde et restauration essayée	Six dossiers de sauvegarde au même endroit, aucun doublon d'un autre : personne ne pouvait dire ce qui était sauvé.
<code>couts</code>	la liste des appels autorisés, le plafond, le geste qui vide la clé	Un script a coûté 32 \$ en une commande — 11 245 appels enchaînés, lancés une fois.
<code>rapport</code>	les gabarits hebdomadaire et panne	Un rapport écrasé perd celui de la semaine d'avant — et c'est celui-là qu'on cherche.
<code>neuf</code>	rien à lui : il pose les autres, après quatre questions	Une version chaînait des kits absents : elle posait la question, on répondait oui, et ça cassait.

Ce que le kit `projet` pose exactement

DOSSIER	CE QU'IL PORTE
<code>_ops/hub/</code>	le générateur, et les quatre chapitres de méthode
<code>_ops/tests/</code>	le banc, et l'anti-régression nommée par date et symptôme
<code>_ops/sabotage/</code>	on casse exprès : la porte doit refuser
<code>_ops/adr/</code>	le gabarit, et le test qui dit s'il faut un ADR
<code>_ops/journaux/</code>	sessions · décisions · pannes, un fichier par mois
<code>_ops/rapports/</code>	datés, jamais écrasés
<code>.githubhooks/</code>	les portes — et <code>core.hooksPath</code> posé, sans quoi elles dorment

➡ **Les gabarits refusent tant qu'ils sont vides, et c'est voulu.** La procédure de déploiement, la charte, la liste des appels payants portent **vos** serveurs, **vos** couleurs, **vos** plafonds. Une valeur par défaut serait suivie.

5 · Les 19 skills

/catalogue # la liste complète, lue à l'instant
/catalogue audit # seulement ce qui concerne l'audit

La forme des réponses — actives tout seules

SKILL	CE QU'ELLE TIENT
focus	résultat en ligne 1, tableaux, questions à la fin, quinze lignes maximum
brainstorm	— ne pas coder sans votre mot. Lire, mesurer, écrire un plan restent libres
plan	écrire le plan dès 3 décisions accumulées, sans demander
cadrage · recadrage	recharger les trois d'un coup quand une longue session a dérivé

Conduire un audit

SKILL	CE QU'ELLE FAIT
audit-code	onze axes, une fiche par constat avec preuve localisée et triple cotation, un vérificateur qui conteste
audit-rapport	le livrable : l'essentiel en page 1, un schéma partout où une phrase peine
audit-correction	un constat, un test rouge, un commit, un test vert. — Ne re-cote rien

★ **Pourquoi trois skills et pas une** : constater, rédiger et réparer ne sont pas le même métier. Celui qui répare cesse de voir ce qu'il vient d'écrire.

Écrire et montrer

SKILL	CE QU'ELLE FAIT
hub	la page noire de contrôle : sa forme, son format de schéma, le contrat d'un onglet
schema-chemins	un seul graphe, plusieurs lectures — des colonnes, un chemin coloré, une pastille d'état
rapporteur	ce qu'un tiers va lire : le résultat en ligne 1, chaque affirmation avec sa commande
balises	baliser les documents pour qu'un agent aille au bon passage au lieu d'avaler le fichier

Le design — et la règle qui a coûté un lot

SKILL	CE QU'ELLE FAIT
charte	★ établir la charte de ce projet : relever une référence, lire ses jetons dans le CSS , mesurer chaque contraste
design- avaliance	la charte du cabinet, mesurée — et le vert va dessous, jamais dessus
veille-design	documenter un site d'inspiration. 🚫 Documenter seulement, jamais implémenter
miroir-site	copier un site publié à l'identique, en statique

🚫 Une couleur relevée à l'écran porte la correction du moniteur, pas la valeur du site. Et un contraste ne s'estime pas : 4,5 pour du texte courant, 3,0 pour l'interface.

Travailler à plusieurs sessions

SKILL	CE QU'ELLE FAIT
mere	arme une session qui surveille et remplit le plan. 🚫 Elle ne code pas, ni ses sous-agents
filles	rattache une session à un domaine — ★ elle ne s'arrête jamais sur une question
installer-mere-filles	la mécanique : pointeuse, rôles, contrat

Une fille qui attend une réponse est une fille morte. C'est la règle qui fait toute la différence entre une équipe et une file d'attente : elle dépose sa question, prévient, et passe au point suivant.

6 · Les principes, et la panne derrière chacun

Aucun de ces principes n'a été inventé. Chacun est le prix payé une fois.

Le principe qui commande tous les autres

Une convention se respecte tant qu'on y pense ; une PORTE, non.

Le verrou de déploiement existait. Il était écrit en majuscules dans un document. Deux sessions l'ont ignoré **de bonne foi** le même soir — quatre collisions en une journée. Le lendemain, le script prenait le verrou lui-même : plus jamais de collision.

Corollaire, et il fait plus de dégâts que le principe : un texte qui décrit un monde disparu est plus dangereux qu'un texte absent, parce qu'on le suit.

Un test vert ne prouve rien tant qu'on ne l'a pas vu rouge

151 témoins verts pendant que la production montrait n'importe quoi. Aucun n'avait jamais été vu rouge.

#	LE RITE, À CHAQUE TEST
1	Écrire le test
2	Casser ce qu'il mesure, constater le rouge
3	Réparer, constater le vert
4	Écrire au commit : « vu rouge en <ce qu'on a cassé> »

➖ C'est le geste que tout le monde saute, parce qu'il consiste à casser volontairement ce qu'on vient de faire marcher. C'est aussi le seul qui distingue un témoin d'une décoration.

Une permission n'est pas une porte

Une cloison entre deux équipes reposait sur des règles de permission. Tout est passé en mode permissif — parce qu'une session qui s'arrête pour demander un clic est une session morte. La cloison a disparu **en une seconde, sans que rien ne le signale**.

MÉCANISME	SURVIT À UN MODE PERMISSIF ?
Absence physique du fichier	✅ absolu
Un hook	✅ il s'exécute quel que soit le mode
Une règle de permission	➖ non — second rideau seulement

Une porte qui ne s'exécute pas est ROUGE

Cinq intégrations continues vertes d'affilée, en 13 à 18 secondes. **Huit portes annoncées, trois exécutées** : les cinq autres sautées avec un simple avertissement. La pastille verte était identique à celle d'un vrai 8 sur 8.

★ **Comptez la durée.** C'est le seul indicateur qui ne se laisse pas maquiller : une intégration qui passe en 17 secondes n'a rien construit. Corollaire dans tout le paquet : « pas de fichier » n'est jamais vert.

Aucun camp ne tient sa propre contrainte

La session qui possédait le test pouvait l'affaiblir pour passer au vert.

CE QUI CONTRAINT	QUI LE POSSÈDE	— JAMAIS
Le banc de tests	celui qui contrôle	celui qui code
Le mur d'isolement	celui qui produit	celui qui contrôle
Les portes	construites par l'un, éprouvées par l'autre	la même session

⚠ Ce n'est pas de la défiance, c'est de l'optique : une vérification faite par celui qui a posé le contrôle **partage ses angles morts**. Il ne verra pas ce qu'il n'a pas pensé à voir. Et ça vaut contre soi-même : un agent de bonne foi, bloqué à 2 h du matin, desserrera la contrainte qu'il peut atteindre.

La raison disparaît toujours avant le code

Le code dit **quoi**. Il ne dit jamais **pourquoi**. Dans six mois, il ne reste que lui.

Ne demandez à personne de « documenter ». Personne n'a jamais commencé parce qu'on le lui a demandé. Deux choses marchent, et elles sont mécaniques : écrire coûte **un seul geste**, et la documentation **se génère**.

Rien n'est déployé tant que le serveur ne SERT pas

— NE PROUVE RIEN	✓ PROUVE
un push, un tag, un « c'est déployé »	une empreinte lue dans la réponse du serveur
« le cache va se vider »	la réponse, maintenant, cache contourné

★ Et le retour arrière s'essaie **avant** d'en avoir besoin. Sa durée mesurée est le temps que durera la panne le jour venu.

Une sauvegarde jamais restaurée n'est pas une sauvegarde

C'est une intention. On découvre qu'elle n'en était pas une le jour où on en a besoin. ⚠ Et deux copies sur le même disque font **une** copie.

Aucune commande ne consomme une clé payante sans un oui pour CETTE exécution

Un script a coûté **32 \$ en une commande** — 11 245 appels enchaînés.

— Ce n'était pas un problème de prudence. Le script était raisonnable, son auteur aussi. Ce qui manquait, c'est quelque chose qui **refuse**.

7 · Le hub — la page que l'on montre

C'est le seul livrable qu'un non-développeur ouvre. Tant que la qualité est un chiffre dans une intégration continue, elle n'existe que pour les développeurs. Dès qu'elle est une page, elle devient un sujet de réunion.

```
node _ops/hub/generer-hub.mjs
```

Ce qu'il montre, et d'où ça vient

CE QUE L'ON VOIT	LU DANS
Ce qui reste à faire	le dernier plan daté
Qui travaille, et depuis quand	<code>_ops/etat/</code> — écrit par la pointeuse
Ce qui attend une décision	<code>_ops/questions/</code>
Vert ou rouge	la dernière sortie du banc de tests
Ce qui a été livré	les commits, les tags

Les quatre chapitres de méthode, dans le hub

ONGLET	CE QU'IL RÉPOND
Coder proprement	journaux, ADR, le commentaire qui dit <i>pourquoi</i> — ★ le seul chapitre qu'on ne rattrape pas
Comment auditer	les sept étapes, et comment on sait qu'on a audité
Anti-régression	installer un filet sur du code déjà en production
Mettre à niveau	annoter un code illisible sans qu'une instruction bouge — et la porte qui le prouve

★ Les quatre répondent à la même question : **comment on sait**. Chacun finit par sept questions qui se répondent par un chiffre, une date ou une ligne de commit — jamais par « oui ».

Deux sorties, un seul générateur

SORTIE	OÙ ELLE S'OUVRE
<code>_ops/hub/index.html</code>	en local, double-clic
<code>_ops/hub/README.md</code>	à l'adresse du dépôt — rendu par GitHub même en privé , lisible du téléphone

🚫 **Le hub ne se rédige pas.** Le `pre-commit` le régénère et l'ajoute au commit : il ne *peut* pas se périmer. Rédigé à la main, il mentirait en deux jours — et on le croirait, puisqu'il est dans le dépôt.

Ce qu'un onglet montre en premier

★ **Ce qui manque, pas ce qui existe.** Compter ne prouve rien : un audit de trente constats sans preuve vaut moins que trois prouvés, et vingt sauvegardes jamais restaurées ne sont pas des sauvegardes.

8 · Ce qui est déjà dans Claude Code

Le chapitre le plus rentable de ce guide. Une bonne part de ce qu'on va chercher sur GitHub est déjà livré. Le savoir fait gagner une semaine, et ça ne coûte rien.

⚠ Cette liste dépend de la version installée. 🚫 N'annoncez jamais une capacité sans l'avoir vue sur votre machine : `/help` et `/plugin`.

FAMILLE	CE QUI EST FOURNI	CE QUE ÇA REMPLACE
Documents	PDF, Word, Excel, PowerPoint — lire, produire, fusionner, remplir un formulaire	tout un pan d'outils payants
Visuel	graphiques accessibles, pages et schémas qui tiennent en clair comme en sombre	les bibliothèques de graphiques
Qualité	relecture de code, revue de sécurité, simplification	une partie des greffons de test
Méthode	comment écrire une skill, consolider une mémoire	les tutoriels « écrire un agent »
Automatisation	boucles, tâches planifiées, et surtout les hooks	les routines maison
Référence	l'API, les modèles, les tarifs, l'écriture de workflows	la documentation qu'on cherche en ligne

La plus sous-estimée

Le réglage des **hooks**. « À chaque fois que X, fais Y » n'est pas une préférence à écrire dans un fichier : c'est un hook, et c'est le harnais qui l'exécute, pas le modèle.

Une règle qui dépend de la bonne volonté d'un agent fatigué ne tiendra pas la nuit. C'est exactement le principe des portes, appliqué à Claude lui-même.

Ce que la relecture fournie ne fait pas

Elle juge la **qualité**. Elle ne fait ni le rite du témoin rouge, ni le sabotage des portes, ni le registre de constats avec preuve localisée, ni la contestation par un autre modèle. Les deux se complètent ; aucun ne remplace l'autre.

9 · Ce qui n'est pas à nous

🚫 **Rien de ce chapitre n'est livré avec ce plugin.** Ce sont les dépôts d'autres personnes. On donne le lien et **la décision à copier** — pas le code.

★ Le plus grand danger de cette page est de tout installer. Empiler quatre outils qui se recoupent donne un dossier que personne ne relit. Un seul, choisi, tenu six mois, vaut plus que quatre découverts. 🙏 Et avant de réutiliser une ligne de leur code dans quelque chose que vous vendez : vérifiez la licence. Pas de licence déclarée = on ne touche pas.

Si votre méthode tient déjà — trois dépôts, un à la fois

DÉPÔT	LA DÉCISION À COPIER
Spec Kit (GitHub)	Séparer la spécification (quoi, pourquoi) du plan (comment, avec quelle pile). ★ Un agent à qui on donne un plan sans spécification invente le besoin , et on le découvre à la livraison.
qa-plugin	Trois garde-fous : un format unique pour tous les bugs · une règle de preuve — pas de capture et d'étapes de reproduction, bug jeté · une porte de vérification : ce qui ne se reproduit pas est éliminé. 🚫 Sa commande de correction modifie le code : chez un client, jamais.
Playwright (Microsoft, Apache 2.0)	★ Playwright = les mains, le modèle = le cerveau. L'IA décide quoi tester et écrit les tests ; l'outil les exécute. 🚫 Un outil ne se refait pas.

Si vous fabriquez vos propres outils

DÉPÔT	CE QU'ON EN RETIENT
OpenSpec	la même idée que Spec Kit, plus légère, et viable sur un projet existant
Superpowers	s'installe en plugin, sans terminal — le parcours d'installation à imiter
ECC	un catalogue d'agents à lire. 🚫 Ne pas tout installer
BMAD-METHOD	des prompts de rôles à lire ; l'architecture est trop lourde pour une petite équipe
anthropics/skills	★ le dépôt officiel : comment bien écrire une skill
Stryker, mutmut	chiffrer la solidité d'un banc : le pourcentage de sabotages attrapés

🚫 **Deux avertissements.** N'installez un dépôt populaire que depuis son compte officiel : les copies sont un vecteur de logiciel malveillant connu. Et ne citez jamais un chiffre de popularité sans avoir ouvert la page — un chiffre invérifiable détruit la crédibilité de tout ce que vous dites d'autre.

Sous-agent ou session séparée ?

	QUAND	COMMENT IL COMMUNIQUE
Sous-agent	une tâche du même travail, avec un contexte propre	il rend son résultat à l'orchestrateur
Session séparée	un travail indépendant , à son rythme	★ par fichiers

★ Et la règle qui ne se négocie pas : le contrôle qualité est un sous-agent **indépendant**, qui peut **refuser**. 🚫 Jamais le même agent qui écrit puis se relit.

10 · Quand ça ne marche pas

L'installation

LE SYMPTÔME	LA CAUSE, PRESQUE TOUJOURS	CE QUE VOUS FAITES
« Repository not found »	l'invitation n'a pas été acceptée, ou le compte n'est pas dans la bonne équipe	acceptez l'invitation sur GitHub
Les commandes n'apparaissent pas	une session déjà ouverte ne voit pas un plugin neuf	rouvrez la session, puis <code>/help</code>
<code>git</code> introuvable après l'avoir installé	★ le <code>PATH</code> de la session a été lu à son ouverture	fermez et rouvrez Claude. 🚫 Ne réinstallez pas

Les portes

LE SYMPTÔME	LA CAUSE	CE QUE VOUS FAITES
Aucune porte ne refuse rien	★ <code>core.hooksPath</code> n'est pas posé : les hooks sont des fichiers morts	<code>git config core.hooksPath .githubhooks</code>
Le sabotage dit qu'une porte dort	elle est là, mais rien ne l'appelle	un dépôt n'a qu'un <code>pre-commit</code> : il doit appeler les autres
Une porte refuse un geste légitime	elle est trop large	🚫 une porte qui refuse tout ne protège rien : corrigez-la, ne la retirez pas

🚫 Jamais de contournement pour passer une porte. Si elle gêne, c'est qu'elle a attrapé quelque chose.

Le hub

LE SYMPTÔME	LA CAUSE	CE QUE VOUS FAITES
Les tests affichent « jamais »	★ aucune mesure n'a eu lieu — et ce n'est pas vert	lancez le banc
Le hub ne dit pas la vérité	🚫 quelqu'un l'a écrit à la main	régénérez-le : ce fichier ne se rédige pas
Un onglet affiche une erreur	un module est cassé	★ c'est voulu : l'erreur s'affiche dans son onglet au lieu d'emporter la page

La forme des réponses

LE SYMPTÔME	LA CAUSE	CE QUE VOUS FAITES
Les réponses redeviennent longues	les trois modes ne sont pas chargés à chaque session	vérifiez le bloc de cadrage dans votre <code>CLAUDE.md</code>
Ça tient trois messages puis ça dérive	★ ce qui tombe d'abord, c'est la règle la plus basse de la liste	<code>/recadrage</code> , et remontez la règle qui saute
Claude code alors que vous réfléchissiez	<code>brainstorm</code> n'est pas chargé	<code>/cadrage</code>

Les trois réflexes, quand rien ne colle

#
1  Trois tours de « toujours cassé » → stop. Nommez l'hypothèse fausse, posez une question
2  Mesurez au lieu de supposer. Un instrument qui rend 0 n'a pas mesuré zéro — il a peut-être mesuré le vide
3 Écrivez la panne au journal. La prochaine fois, quelqu'un la trouvera

Une dernière chose. Si une règle de ce document vous gêne, regardez d'abord la panne qui l'a fait naître : elle est écrite à côté. Si elle ne vous concerne pas, retirez la règle — et écrivez pourquoi.

 Ce qu'on ne fait jamais : la contourner en la laissant écrite. Un texte qu'on garde et qu'on n'applique plus est le pire des deux mondes — quelqu'un le suivra.